01 — Data: Download, Tiền xử lý & Augmentation

Chạy đầu tiên. Output: Kaggle Dataset `faidset-processed` chứa:

- `train.csv` — data đã augmented (gốc + back-translation)
- `val.csv`, `test_en.csv`, `test_vi.csv`

Pipeline (theo thứ tự):

```
1. Download FAIDSet (EN+VI) + MAGE từ HuggingFace
2. Tổ chức raw files vào thư mục
3. Load → clean text → filter < 50 ký tự → stratified split →
train.csv / val.csv / test CSVs
4. Back-translate trên train.csv đã clean → clean lại → filter → gộp →
train.csv
5. Upload tất cả lên Kaggle Dataset
```

> **Tại sao augment SAU khi tiền xử lý?** Text gốc đã được clean và filter. Text sau back-translation cũng phải qua đúng pipeline clean+filter đó trước khi gộp vào train. Đảm bảo `train.csv` có chất lượng đồng nhất với `train.csv`.

```
!pip install huggingface_hub langdetect datasets transformers torch
sentencepiece pandas scikit-learn -q

━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 981.5/981.5 kB 15.9 MB/s eta
0:00:00a 0:00:01
etadata (setup.py) ...

import json, re, shutil, os, subprocess
from pathlib import Path
from collections import Counter
from huggingface_hub import hf_hub_download
from langdetect import detect, LangDetectException
from datasets import load_dataset
from sklearn.model_selection import train_test_split
import pandas as pd
import numpy as np
import torch
from transformers import MarianMTModel, MarianTokenizer

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Device: {device}")

BASE      = Path("raw_data")
PROCESSED = Path("processed_data")
TMP       = Path("tmp_download")
```

```
PROCESSED.mkdir(exist_ok=True)
TMP.mkdir(exist_ok=True)

# Số mẫu augment mỗi ngôn ngữ (giảm xuống 500 nếu Kaggle
timeout)
N_AUG_EN = 2000
N_AUG_VI = 2000
MIN_CHARS = 50
```

```
Device: cuda
```

Phần 1 — Download dữ liệu

```
# ── FAIDSet ──────────────────────────────────────────────────────────
# Giữ nguyên logic từ file gốc:
# - get_kind(): đọc label từ JSON (human-written / llm-generated /
collaborative)
# - get_lang(): dùng langdetect.detect() trên text thực — không dùng
field language trong JSON
from langdetect import detect, LangDetectException

for split in ["train", "valid", "test"]:
    for d in ["eng/AI", "eng/human", "vi/AI", "vi/human"]:
        (BASE / split / d).mkdir(parents=True, exist_ok=True)

def get_kind(record):
    label = str(record.get("label", "")).lower()
    label = label.replace("\u2013", "-").replace("\u2014", "-")
    if label == "human-written": return "human"
    if label == "llm-generated":  return "AI"
    return None  # collaborative → bỏ qua

def get_lang(text):
    try:
        lang = detect(text[:500])
        return {"vi": "vi", "en": "eng"}.get(lang)  # chỉ giữ vi và
eng
    except LangDetectException:
        return None

raw_files = {}
for split, filename in {"train": "train.jsonl", "valid":
"valid.jsonl", "test": "test.jsonl"}.items():
    path = hf_hub_download(repo_id="ngocminhta/FAIDSet",
filename=filename,
                           repo_type="dataset", local_dir=str(TMP))
    raw_files[split] = Path(path)
```

```
total_skipped = Counter()
for split, path in raw_files.items():
    counts, skipped = Counter(), Counter()
    with open(path, encoding="utf-8") as f:
        records = [json.loads(l) for l in f if l.strip()]
    print(f"\n[{split.upper()}] {len(records):,} records")
    for idx, record in enumerate(records):
        kind = get_kind(record)
        if not kind:  skipped["collaborative"] += 1; continue
        text = record.get("text", "").strip()
        if not text:  skipped["empty"] += 1;         continue
        lang = get_lang(text)
        if not lang:  skipped["other_lang"] += 1;    continue
        fname = f"{split}_{idx:06d}.txt"
        (BASE / split / lang / kind / fname).write_text(text,
encoding="utf-8")
        counts[(lang, kind)] += 1
    for (lang, kind), c in sorted(counts.items()):
        print(f"  raw_data/{split}/{lang}/{kind}/ → {c:,} files")
    print(f"  Bỏ qua: {dict(skipped)}")
    total_skipped += skipped

shutil.rmtree(TMP)
print(f"\n FAIDSet done! Tổng bỏ qua: {dict(total_skipped)}")
```

{"model_id":"3868dc868dd9463c841fcbe0f952da9f","version_major":2,"version_minor":0}

```
Warning: You are sending unauthenticated requests to the HF Hub.
Please set a HF_TOKEN to enable higher rate limits and faster
downloads.
```

{"model_id":"f973771fc33443d897d9d0dba95b1c6a","version_major":2,"version_minor":0}

{"model_id":"c6acfb0522ef467e80b82af048f16670","version_major":2,"version_minor":0}

```
[TRAIN] 60,676 records
  raw_data/train/eng/AI/ → 7,883 files
  raw_data/train/eng/human/ → 5,050 files
  raw_data/train/vi/AI/ → 6,525 files
  raw_data/train/vi/human/ → 9,112 files
  Bỏ qua: {'collaborative': 32091, 'other_lang': 15}

[VALID] 12,502 records
  raw_data/valid/eng/AI/ → 1,272 files
  raw_data/valid/eng/human/ → 1,038 files
  raw_data/valid/vi/AI/ → 1,316 files
```

```
  raw_data/valid/vi/human/ → 1,997 files
  Bỏ qua: {'collaborative': 6876, 'other_lang': 3}

[TEST] 12,505 records
  raw_data/test/eng/AI/ → 1,268 files
  raw_data/test/eng/human/ → 1,074 files
  raw_data/test/vi/AI/ → 1,320 files
  raw_data/test/vi/human/ → 1,959 files
  Bỏ qua: {'collaborative': 6879, 'other_lang': 5}

 FAIDSet done! Tổng bỏ qua: {'collaborative': 45846, 'other_lang': 23}
```

```
# ── MAGE
────────────────────────────────────────────────────────────
def save_to_raw(df_in, base_dir, prefix):
    d = BASE / base_dir
    (d / "AI").mkdir(parents=True, exist_ok=True)
    (d / "human").mkdir(parents=True, exist_ok=True)
    for idx, row in df_in.iterrows():
        kind = "human" if row["label"] == 0 else "AI"
        (d / kind / f"{prefix}_{idx:05d}.txt").write_text(row["text"],
encoding="utf-8")

def load_mage(split_name):
    print(f"Loading MAGE {split_name}...")
    ds = load_dataset("yaful/MAGE", split=split_name)
    df = ds.to_pandas()[["text", "label"]].copy()
    df["label"] = df["label"].apply(lambda x: 0 if x == 1 else 1)  #
đảo: 0=human, 1=AI
    return df[df["text"].str.len() >=
MIN_CHARS].reset_index(drop=True)

df_mage_test = load_mage("test")
save_to_raw(df_mage_test, "mage_test", "magetest")
print(f"MAGE test: {len(df_mage_test):,}")

df_mage_train = load_mage("train")
df_mage_sample = pd.concat([
    df_mage_train[df_mage_train["label"]==0].sample(7500,
random_state=42),
    df_mage_train[df_mage_train["label"]==1].sample(7500,
random_state=42)
]).reset_index(drop=True)
save_to_raw(df_mage_sample, "mage_train", "magetrain")
print(f"MAGE train sample: {len(df_mage_sample):,}")
print("\n MAGE done!")
```

```
Loading MAGE test...
```

```
{"model_id":"3a68206143254c629fadb2fbe19eb8ec","version_major":2,"version_minor":0}

{"model_id":"0500d5fe8a774d0d85664b03f30fc2ca","version_major":2,"version_minor":0}

{"model_id":"bed3bf281a9247efb8574703ab2d9769","version_major":2,"version_minor":0}

{"model_id":"20e625aee8064bf2b1f43a12f5bceac1","version_major":2,"version_minor":0}

{"model_id":"fd97d13093884d92b441936c1d6a12d7","version_major":2,"version_minor":0}

{"model_id":"596a761d7b9b490fad9df1b03e1032a8","version_major":2,"version_minor":0}

{"model_id":"00b1ddceb76649b0b3f1d91f1608c7fa","version_major":2,"version_minor":0}

{"model_id":"c3be64100a094daba9f234755c80d72a","version_major":2,"version_minor":0}

{"model_id":"30693b5d3be144b0819064dcccd2f3e8","version_major":2,"version_minor":0}

MAGE test: 60,279
Loading MAGE train...
MAGE train sample: 15,000

 MAGE done!
```

Phần 2 — Extract: gộp files thành nhóm

```
for kind in ["AI", "human"]:
    for d in ["train_all/eng", "train_all/vi", "test_en", "test_vi"]:
        (BASE / d / kind).mkdir(parents=True, exist_ok=True)

# train_all = FAIDSet train+valid + MAGE train 15k
for split in ["train", "valid"]:
    for lang in ["eng", "vi"]:
        for kind in ["AI", "human"]:
            src = BASE / split / lang / kind
            if src.exists():
                for f in src.glob("*.txt"):
                    shutil.copy2(f, BASE / "train_all" / lang / kind / 
f"{split}_{f.name}")
```

```
for kind in ["AI", "human"]:
    src = BASE / "mage_train" / kind
    if src.exists():
        for f in src.glob("*.txt"):
            shutil.copy2(f, BASE / "train_all" / "eng" / kind /
f.name)

# test_en = FAIDSet test EN + MAGE test
for kind in ["AI", "human"]:
    for src in [BASE / "test" / "eng" / kind, BASE / "mage_test" /
kind]:
        if src.exists():
            for f in src.glob("*.txt"):
                shutil.copy2(f, BASE / "test_en" / kind / f.name)

# test_vi = FAIDSet test VI
for kind in ["AI", "human"]:
    src = BASE / "test" / "vi" / kind
    if src.exists():
        for f in src.glob("*.txt"):
            shutil.copy2(f, BASE / "test_vi" / kind / f.name)

print("Extract xong:")
for folder in ["train_all/eng", "train_all/vi", "test_en", "test_vi"]:
    for kind in ["AI", "human"]:
        d = BASE / folder / kind
        if d.exists():
            print(f"  {folder}/{kind}/  →
{len(list(d.glob('*.txt'))):,} files")
```

```
Extract xong:
  train_all/eng/AI/  →  16,655 files
  train_all/eng/human/  →  13,588 files
  train_all/vi/AI/  →  7,841 files
  train_all/vi/human/  →  11,109 files
  test_en/AI/  →  31,722 files
  test_en/human/  →  30,899 files
  test_vi/AI/  →  1,320 files
  test_vi/human/  →  1,959 files
```

Phần 3 — Load → Clean → Filter → Split → lưu train.csv / val.csv / test CSVs

Hàm `clean_text` sẽ được tái sử dụng ở Phần 4 để đảm bảo augmented text đi qua **đúng cùng một pipeline** với data gốc.

```python
# ── Hàm dùng chung cho cả data gốc lẫn augmented ──
def clean_text(text):
    """Chuẩn hóa whitespace. Dùng chung cho data gốc VÀ
augmented."""
    # 2 regex giống hệt file gốc
    text = str(text)
    text = re.sub(r"\s+", " ", text)
    text = re.sub(r"[^\S\n]+", " ", text)
    return text.strip()

def clean_and_filter(df, name):
    """Clean text + filter quá ngắn. Dùng chung cho data gốc VÀ
augmented."""
    df = df.copy()
    df["text"] = df["text"].apply(clean_text)
    before = len(df)
    df = df[df["text"].str.len() >= MIN_CHARS].reset_index(drop=True)
    print(f"[{name}] {before:,} → {len(df):,} (bỏ {before - len(df)}
quá ngắn)")
    return df


# ── Load raw files ──
FOLDER_MAP = {
    ("eng", "AI"):    {"language": "en", "label": 1},
    ("eng", "human"): {"language": "en", "label": 0},
    ("vi",  "AI"):    {"language": "vi", "label": 1},
    ("vi",  "human"): {"language": "vi", "label": 0},
}

def load_folder(base_folder):
    records = []
    for (lang, kind), meta in FOLDER_MAP.items():
        folder = base_folder / lang / kind
        if not folder.exists(): continue
        for f in folder.glob("*.txt"):
            records.append({"text": f.read_text(encoding="utf-8"),
**meta})
    return pd.DataFrame(records)

def load_flat(folder, language):
    records = []
    for kind, label in [("AI", 1), ("human", 0)]:
        d = folder / kind
        if not d.exists(): continue
        for f in d.glob("*.txt"):
            records.append({"text": f.read_text(encoding="utf-8"),
                            "language": language, "label": label})
    return pd.DataFrame(records)
```

```python
# ── Clean + filter ──
df_train_raw = clean_and_filter(load_folder(BASE / "train_all"),
"TRAIN ALL")
df_test_en   = clean_and_filter(load_flat(BASE / "test_en", "en"),
"TEST EN")
df_test_vi   = clean_and_filter(load_flat(BASE / "test_vi", "vi"),
"TEST VI")

print()
print(df_train_raw.groupby(["language", "label"]).size().to_string())


# ── Stratified split 90/10 → train.csv / val.csv ──
df_train_raw["stratify_key"] = df_train_raw["language"] + "_" +
df_train_raw["label"].astype(str)
df_train, df_val = train_test_split(
    df_train_raw, test_size=0.1, random_state=42,
    stratify=df_train_raw["stratify_key"]
)
df_train =
df_train.drop(columns=["stratify_key"]).reset_index(drop=True)
df_val   =
df_val.drop(columns=["stratify_key"]).reset_index(drop=True)

df_val.to_csv  (PROCESSED / "val.csv",      index=False,
encoding="utf-8-sig")
df_test_en.to_csv(PROCESSED / "test_en.csv", index=False,
encoding="utf-8-sig")
df_test_vi.to_csv(PROCESSED / "test_vi.csv", index=False,
encoding="utf-8-sig")

print(f"\nVal + test lưu xong (train.csv sẽ lưu sau augmentation):")
print(f"  val.csv      → {len(df_val):,}")
print(f"  test_en.csv  → {len(df_test_en):,}")
print(f"  test_vi.csv  → {len(df_test_vi):,}")
```

```
[TRAIN ALL] 49,193 → 49,193 (bỏ 0 quá ngắn)
[TEST EN] 62,621 → 62,621 (bỏ 0 quá ngắn)
[TEST VI] 3,279 → 3,278 (bỏ 1 quá ngắn)

language  label
en        0        13588
          1        16655
vi        0        11109
          1         7841

Val + test lưu xong (train.csv sẽ lưu sau augmentation):
  val.csv      → 4,920
```

```
test_en.csv  → 62,621
test_vi.csv  → 3,278
```

Phần 4 — Back-translation Augmentation

Augment trực tiếp từ `df_train` (đã clean+filter ở Phần 3). Text sau dịch đi qua **đúng cùng hàm `clean_and_filter`** như data gốc trước khi gộp vào `train_aug.csv`.

- EN → FR → EN dùng `Helsinki-NLP/opus-mt-en-fr` + `opus-mt-fr-en`
- VI → EN → VI dùng `Helsinki-NLP/opus-mt-vi-en` + `opus-mt-en-vi`
- Chỉ augment Human (label=0) — class bị FP nhiều nhất (27k lỗi ở baseline)

```
class BackTranslator:
    def __init__(self, src_lang, mid_lang, device):
        self.device = device
        fwd = f"Helsinki-NLP/opus-mt-{src_lang}-{mid_lang}"
        bwd = f"Helsinki-NLP/opus-mt-{mid_lang}-{src_lang}"
        print(f"  Loading {fwd}...")
        self.tok_fwd   = MarianTokenizer.from_pretrained(fwd)
        self.model_fwd =
MarianMTModel.from_pretrained(fwd).to(device).eval()
        print(f"  Loading {bwd}...")
        self.tok_bwd   = MarianTokenizer.from_pretrained(bwd)
        self.model_bwd =
MarianMTModel.from_pretrained(bwd).to(device).eval()
        print(f"  Ready: {src_lang}→{mid_lang}→{src_lang}")

    def _translate(self, texts, model, tokenizer, batch_size=16):
        results = []
        for i in range(0, len(texts), batch_size):
            batch = texts[i:i+batch_size]
            enc = tokenizer(batch, return_tensors="pt", padding=True,
                            truncation=True,
max_length=256).to(self.device)
            with torch.no_grad():
                out = model.generate(**enc, num_beams=4,
max_new_tokens=256)
            results.extend(tokenizer.batch_decode(out,
skip_special_tokens=True))
        return results

    def augment(self, texts):
        mid  = self._translate(texts, self.model_fwd, self.tok_fwd)
        back = self._translate(mid,   self.model_bwd, self.tok_bwd)
        return back

    def free(self):
```

```python
        del self.model_fwd, self.model_bwd
        torch.cuda.empty_cache()

# Chọn mẫu Human từ df_train (đã clean+filter)
en_human = df_train[(df_train["language"]=="en") &
(df_train["label"]==0)].sample(N_AUG_EN, random_state=42)
vi_human = df_train[(df_train["language"]=="vi") &
(df_train["label"]==0)].sample(N_AUG_VI, random_state=42)
print(f"Sẽ augment: {len(en_human)} EN human + {len(vi_human)} VI
human")
```

```
Sẽ augment: 2000 EN human + 2000 VI human
```

```python
# EN → FR → EN
print("Augmenting EN...")
bt_en = BackTranslator("en", "fr", device)
en_aug_raw = bt_en.augment(en_human["text"].tolist())
bt_en.free()

# Đưa qua ĐÚNG hàm clean_and_filter như data gốc
en_aug_df = clean_and_filter(
    pd.DataFrame({"text": en_aug_raw, "label": 0, "language": "en"}),
    "EN AUG"
)
print(f"EN sau augment+clean: {len(en_aug_df)} mẫu")
```

```
Augmenting EN...
  Loading Helsinki-NLP/opus-mt-en-fr...
```


```
{"model_id":"06ba33dcf76a4167863abd43fd36b9d9","version_major":2,"version_minor":0}

{"model_id":"2e5cdd763a744b23ac555d8168548c85","version_major":2,"version_minor":0}

{"model_id":"647f8b0dc9f54cabb827a6fb4b0ae881","version_major":2,"version_minor":0}

{"model_id":"ec127146c54b4868b7e6f0d53cf3acca","version_major":2,"version_minor":0}
```


```
/usr/local/lib/python3.12/dist-packages/transformers/models/marian/
tokenization_marian.py:176: UserWarning: Recommended: pip install
sacremoses.
  warnings.warn("Recommended: pip install sacremoses.")
```


```
{"model_id":"bb00553aa1a54a72af289d3c59312439","version_major":2,"version_minor":0}

{"model_id":"c320414a70324cd7a446a86b7ed23a66","version_major":2,"version_minor":0}
```

{"model_id":"e65f32e1fd1342f68deeaf4854a3e4a4","version_major":2,"version_minor":0}

{"model_id":"21bf217c63994e259435b0ae24b7dbe0","version_major":2,"version_minor":0}

```
The tied weights mapping and config for this model specifies to tie
model.shared.weight to model.decoder.embed_tokens.weight, but both are
present in the checkpoints, so we will NOT tie them. You should update
the config with `tie_word_embeddings=False` to silence this warning
The tied weights mapping and config for this model specifies to tie
model.shared.weight to model.encoder.embed_tokens.weight, but both are
present in the checkpoints, so we will NOT tie them. You should update
the config with `tie_word_embeddings=False` to silence this warning
```

{"model_id":"d17ad9da6e5343baae09c6be16b69c58","version_major":2,"version_minor":0}

```
  Loading Helsinki-NLP/opus-mt-fr-en...
```

{"model_id":"a7fbff4855ec4427b8c6cbe75450a5d2","version_major":2,"version_minor":0}

{"model_id":"bf0cc989729148049445ba2d954be72d","version_major":2,"version_minor":0}

{"model_id":"6870e5a1ff1f4070b66e9fc77993ff3b","version_major":2,"version_minor":0}

{"model_id":"a1b43c871c7549cc8941ec5cf888bfeb","version_major":2,"version_minor":0}

{"model_id":"00cc4010fad74c18a36ce8ec19310e20","version_major":2,"version_minor":0}

{"model_id":"71754ce113284f57bb6a1159d1bb52d9","version_major":2,"version_minor":0}

{"model_id":"20940a7f589e4ca9933457902057586e","version_major":2,"version_minor":0}

{"model_id":"31467216d2cf4281b648f40aaec37f9d","version_major":2,"version_minor":0}

```
  Ready: en→fr→en
[EN AUG] 2,000 → 1,998 (bỏ 2 quá ngắn)
EN sau augment+clean: 1998 mẫu
```

```
# VI → EN → VI
print("Augmenting VI...")
bt_vi = BackTranslator("vi", "en", device)
vi_aug_raw = bt_vi.augment(vi_human["text"].tolist())
```

```python
bt_vi.free()

# Đưa qua ĐÚNG hàm clean_and_filter như data gốc
vi_aug_df = clean_and_filter(
    pd.DataFrame({"text": vi_aug_raw, "label": 0, "language": "vi"}),
    "VI AUG"
)
print(f"VI sau augment+clean: {len(vi_aug_df)} mẫu")
```

```
Augmenting VI...
  Loading Helsinki-NLP/opus-mt-vi-en...
```


```
{"model_id":"0a78fb3ed90b42778e006c806300fb1f","version_major":2,"version_minor":0}

{"model_id":"302f463d32304f91a7b6b32a62f10d2d","version_major":2,"version_minor":0}

{"model_id":"dd900851bdd84fa6adcc6fc1a732d5d8","version_major":2,"version_minor":0}

{"model_id":"dffc23abcb08480182276abc55e1ce6c","version_major":2,"version_minor":0}
```


```
/usr/local/lib/python3.12/dist-packages/transformers/models/marian/tokenization_marian.py:176: UserWarning: Recommended: pip install sacremoses.
  warnings.warn("Recommended: pip install sacremoses.")
```


```
{"model_id":"780e5b5c8e2044949ecb8831871532bd","version_major":2,"version_minor":0}

{"model_id":"b04a612dadaf47c3861d3740e7f3b753","version_major":2,"version_minor":0}

{"model_id":"512f700cff644222abdfbcce8b94812b","version_major":2,"version_minor":0}

{"model_id":"74934df8d2a6423780d7b9b7d12a5f87","version_major":2,"version_minor":0}
```


```
The tied weights mapping and config for this model specifies to tie model.shared.weight to model.decoder.embed_tokens.weight, but both are present in the checkpoints, so we will NOT tie them. You should update the config with `tie_word_embeddings=False` to silence this warning
The tied weights mapping and config for this model specifies to tie model.shared.weight to model.encoder.embed_tokens.weight, but both are present in the checkpoints, so we will NOT tie them. You should update the config with `tie_word_embeddings=False` to silence this warning
```


```
{"model_id":"565a2fa45cec49abb1517c8365cf5034","version_major":2,"version_minor":0}
```

```
  Loading Helsinki-NLP/opus-mt-en-vi...
```


{"model_id":"42eb2252fadd4957ac766f49fece4858","version_major":2,"version_minor":0}

{"model_id":"77649419a21f4771b6d48c12d084cf9c","version_major":2,"version_minor":0}

{"model_id":"5a103c120ab3414dbb1fe870c57e8bef","version_major":2,"version_minor":0}

{"model_id":"0f4ed10707444157b30f8afbaf573a5a","version_major":2,"version_minor":0}

{"model_id":"3f6e208449bc462b8b070b46c01b6de5","version_major":2,"version_minor":0}

{"model_id":"c4262908e8344e0986f21faf4a4a3c34","version_major":2,"version_minor":0}

{"model_id":"2e27cec11b1d415db712f25f458b0fa9","version_major":2,"version_minor":0}


```
The tied weights mapping and config for this model specifies to tie
model.shared.weight to model.decoder.embed_tokens.weight, but both are
present in the checkpoints, so we will NOT tie them. You should update
the config with `tie_word_embeddings=False` to silence this warning
The tied weights mapping and config for this model specifies to tie
model.shared.weight to model.encoder.embed_tokens.weight, but both are
present in the checkpoints, so we will NOT tie them. You should update
the config with `tie_word_embeddings=False` to silence this warning
```


{"model_id":"cfc4410d12344f30898cdaee9b1189a1","version_major":2,"version_minor":0}

{"model_id":"f69162ceaa6d453eb99fec07cce830d4","version_major":2,"version_minor":0}


```
  Ready: vi→en→vi
[VI AUG] 2,000 → 1,986 (bỏ 14 quá ngắn)
VI sau augment+clean: 1986 mẫu
```

```
# Gộp: train gốc + augmented → train.csv
# Cả 2 phần đều đã qua cùng clean_and_filter → đảm bảo đồng nhất
aug_df      = pd.concat([en_aug_df, vi_aug_df]).reset_index(drop=True)
train_df = pd.concat([df_train, aug_df]).reset_index(drop=True)

train_df.to_csv(PROCESSED / "train.csv", index=False, encoding="utf-8-sig")
```

```
print(f"\nLưu: train.csv → {len(train_df):,} rows (gốc + augmented)")
print(train_df.groupby(["language", "label"]).size().to_string())
print(f"\nTăng so với train.csv: +{len(train_df) - len(df_train):,}
mẫu")
```

```
Lưu: train.csv → 48,257 rows (gốc + augmented)
language  label
en        0        14227
          1        14989
vi        0        11984
          1         7057

Tăng so với train.csv: +3,984 mẫu
```

Phần 5 — Upload lên Kaggle Dataset

```
dataset_name = "faidset-processed"
kaggle_user  = [l.split(":")[1].strip() for l in
                subprocess.run("kaggle config view", shell=True,
capture_output=True, text=True)
                .stdout.split("\n") if "username" in l][0]

with open(PROCESSED / "dataset-metadata.json", "w") as f:
    json.dump({"title": dataset_name, "id":
f"{kaggle_user}/{dataset_name}",
               "licenses": [{"name": "CC0-1.0"}]}, f)

check = subprocess.run(
    f"kaggle datasets list --user {kaggle_user} --search
{dataset_name}",
    shell=True, capture_output=True, text=True
)
cmd = (f'kaggle datasets version -p {PROCESSED} -m "update"'
       if dataset_name in check.stdout
       else f"kaggle datasets create -p {PROCESSED}")
result = subprocess.run(cmd, shell=True, capture_output=True,
text=True)
print(result.stdout or result.stderr)
print(f"\nDone! Dataset: {kaggle_user}/{dataset_name}")
print(f"Files: train.csv, val.csv, test_en.csv, test_vi.csv")
```

```
Warning: Looks like you're using an outdated `kaggle` version
(installed: 2.0.1), please consider upgrading to the latest version
(2.0.2)
Starting upload for file train.csv
Upload successful: train.csv (46MB)
Starting upload for file test_en.csv
```

```
Upload successful: test_en.csv (75MB)
Starting upload for file test_vi.csv
Upload successful: test_vi.csv (3MB)
Starting upload for file val.csv
Upload successful: val.csv (5MB)
Dataset version is being created. Please check progress at
https://www.kaggle.com/datasets/cminhnguyndsdsds/faidset-processed


Done! Dataset: cminhnguyndsdsds/faidset-processed
Files: train.csv, val.csv, test_en.csv, test_vi.csv
```